

Implementação Micro-NAS: Comparação das SSt em Ambiente Controlado

Gabriel Brum

2 de maio de 2026

Conteúdo

1	Introdução à Implementação	2
2	Decisões de Projeto	2
2.1	Ferramentas e Bibliotecas	2
2.2	Dataset: MNIST em Subconjunto Reduzido	2
3	Definição do SSp e da VSt	3
3.1	Search Space (SSp)	3
3.2	Validation Strategy (VSt)	3
4	Implementação das SSt	4
4.1	Random Search (RS)	4
4.2	Evolutionary Algorithm (EA)	4
4.3	Reinforcement Learning (RL) — Multi-Armed Bandit	5
4.4	Bayesian Optimization (BO)	6
4.5	Hill Climbing / Gradient-based (GO)	7
5	Protocolo de Comparação	8
6	Limitações da Implementação	8
A	Introdução e Fundamentação Teórica	10

1 Introdução à Implementação

Esta seção documenta a implementação prática desenvolvida para o seminário, cujo objetivo é demonstrar o funcionamento das cinco Estratégias de Busca (SSt) mais recorrentes na literatura de *Neural Architecture Search* (NAS) em um ambiente controlado e de baixo custo computacional.

A implementação foi concebida como um **Micro-NAS**: um experimento simplificado que preserva a lógica central de cada SSt, mas opera sobre um espaço de busca (SSp) pequeno e uma estratégia de validação (VSt) de baixa fidelidade. Isso permite executar e comparar todos os algoritmos em minutos, sem necessidade de infraestrutura especializada, tornando-o adequado para fins didáticos.

O código foi desenvolvido em Python, executado em ambiente Google Colab com GPU T4, e está disponível no notebook `Micro_NAS_revisado.ipynb` entregue junto a este relatório.

2 Decisões de Projeto

2.1 Ferramentas e Bibliotecas

A implementação utiliza apenas bibliotecas de propósito geral, sem frameworks NAS dedicados (como NNI, Auto-Keras ou Optuna). Essa escolha é intencional: o objetivo é expor o mecanismo de cada SSt, não abstraí-lo.

- **TensorFlow / Keras**: construção, compilação e treinamento dos modelos candidatos. Fornece a VSt completa em poucas linhas.
- **NumPy**: operações vetoriais auxiliares (argmax sobre previsões do surrogate, normalização de dados).
- **Scikit-learn** (RandomForestRegressor): modelo substituto (*surrogate model*) para a Otimização Bayesiana.
- **Módulos padrão** (random, time): amostragem, mutação e medição de tempo de execução.

2.2 Dataset: MNIST em Subconjunto Reduzido

O MNIST foi escolhido por ser um *benchmark* clássico, de carregamento imediato via Keras e com imagens suficientemente simples para que 1 época de treinamento já produza um sinal de desempenho discriminativo entre arquiteturas.

Para reduzir o custo de cada avaliação (VSt de baixa fidelidade), foram utilizados apenas:

- **3.000 amostras de treino** (em vez de 60.000)
- **1.000 amostras de validação** (em vez de 10.000)

As imagens foram normalizadas para $[0, 1]$ e expandidas para o formato $(28 \times 28 \times 1)$ exigido pela camada Conv2D.

3 Definição do SSp e da VSt

3.1 Search Space (SSp)

O SSp é do tipo *layer-based*: a topologia da rede é fixa (sequencial) e a busca ocorre exclusivamente sobre os valores de quatro hiperparâmetros por camada. Isso resulta em $3 \times 2 \times 3 \times 3 = 54$ arquiteturas distintas possíveis.

Tabela 1: Hiperparâmetros e valores do SSp.

Hiperparâmetro	Valores possíveis	Justificativa
filters	16, 32, 64	Capacidade da camada convolucional
kernel_size	3, 5	Tamanho do campo receptivo
activation	relu, elu, tanh	Funções de ativação mais comuns
dropout	0,1 0,3 0,5	Regularização da camada densa

A arquitetura resultante para um conjunto de parâmetros θ é sempre:

Input(28×28×1) → Conv2D(θ) → MaxPool(2×2) → Flatten → Dropout(θ) →
Dense(10, softmax)

3.2 Validation Strategy (VSt)

A VSt implementada é do tipo *Partial Training* (Tipo 1): cada arquitetura candidata é treinada do zero por exatamente **1 época** no subconjunto reduzido, e a acurácia de validação ao final dessa época é retornada como sinal de desempenho para a SSt.

```
1 def avaliar_arquitetura(params):
2     model = models.Sequential([
3         Input(shape=(28, 28, 1)),
4         layers.Conv2D(params['filters'], params['kernel_size'],
5                       activation=params['activation']),
6         layers.MaxPooling2D((2, 2)),
7         layers.Flatten(),
8         layers.Dropout(params['dropout']),
9         layers.Dense(10, activation='softmax')
10    ])
11    model.compile(optimizer='adam',
12                 loss='sparse_categorical_crossentropy',
13                 metrics=['accuracy'])
14    history = model.fit(x_train, y_train, epochs=1,
15                      validation_data=(x_test, y_test),
16                      verbose=0)
17    return history.history['val_accuracy'][0]
```

Listing 1: Implementação da VSt (Partial Training, 1 época).

Simplificações em relação à literatura:

- Nenhum *weight sharing* entre candidatos (o tipo 2 de Partial Training).
- O número fixo de 1 época não se adapta à curva de aprendizado (o que caracterizaria *Adaptive Training*).
- Não há *early stopping*, métricas de zero-shot, nem preditor de desempenho.

4 Implementação das SSt

Cada SSt recebe como entrada o orçamento `max_evals` (número máximo de chamadas reais à VSt) e retorna a melhor arquitetura encontrada junto com sua acurácia de validação. O orçamento foi fixado em **10 avaliações** para todos os algoritmos.

4.1 Random Search (RS)

Mecanismo: a cada iteração, uma arquitetura é amostrada de forma independente e uniforme do SSp, sem usar nenhuma informação das avaliações anteriores. É o *baseline* da comparação.

Simplificação: o RS da literatura frequentemente é usado com um protocolo de *hyperband* ou *successive halving* para alocar orçamentos variáveis de treinamento. Aqui, cada avaliação tem custo fixo (1 época).

```
1 def random_search(max_evals):
2     melhor_acc, melhor_arq = 0, None
3     for _ in range(max_evals):
4         arq = gerar_aleatorio()
5         acc = avaliar_arquitetura(arq)
6         if acc > melhor_acc:
7             melhor_acc, melhor_arq = acc, arq
8     return melhor_arq, melhor_acc
```

Listing 2: Random Search.

4.2 Evolutionary Algorithm (EA)

Mecanismo: mantém uma população de arquiteturas avaliadas. A cada geração, seleciona os dois melhores indivíduos como pais, aplica *crossover* uniforme (cada gene vem do pai ou da mãe com probabilidade 0,5) e, com probabilidade 0,3, aplica uma *mutação* em um gene aleatório. O filho gerado é avaliado e adicionado à população.

Simplificações em relação à literatura:

- A população nunca descarta indivíduos (cresce indefinidamente) — algoritmos como AmoebaNet usam envelhecimento e eliminação dos mais antigos.
- A seleção é sempre determinística (os 2 melhores), sem torneio ou seleção proporcional à *fitness*.
- Sem operadores de crossover mais sofisticados (ponto de corte único, multi-ponto, etc.).
- A população inicial é de apenas 2 indivíduos para respeitar o orçamento de 10 avaliações.

Bug corrigido na revisão: a implementação original gerava a arquitetura e a acurácia do indivíduo inicial de forma desacoplada (dois `gerar_aleatorio()` separados), associando a acurácia de uma arquitetura à representação de outra. A correção garante que a arquitetura armazenada e a avaliada sejam a mesma instância.

```
1 def evolutionary_search(max_evals):
2     populacao = []
3     for _ in range(2):
4         arq = gerar_aleatorio()
5         acc = avaliar_arquitetura(arq) # avalia a MESMA
            arquitetura gerada
```

```

6     populacao.append({'arq': arq, 'acc': acc})
7     evals = 2
8
9     while evals < max_evals:
10        populacao.sort(key=lambda x: x['acc'], reverse=True)
11        pai, mae = populacao[0], populacao[1]
12
13        # Crossover uniforme
14        filho_arq = {
15            k: pai['arq'][k] if random.random() > 0.5 else
16                mae['arq'][k]
17            for k in search_space
18        }
19        # Mutacao com prob. 0.3
20        if random.random() < 0.3:
21            gene = random.choice(list(search_space.keys()))
22            filho_arq[gene] = random.choice(search_space[gene])
23
24        acc = avaliar_arquitetura(filho_arq)
25        populacao.append({'arq': filho_arq, 'acc': acc})
26        evals += 1
27
28    populacao.sort(key=lambda x: x['acc'], reverse=True)
29    return populacao[0]['arq'], populacao[0]['acc']

```

Listing 3: Evolutionary Algorithm.

4.3 Reinforcement Learning (RL) — Multi-Armed Bandit

Mecanismo: modela cada hiperparâmetro como um *multi-armed bandit* independente. O agente mantém uma distribuição de probabilidades sobre os valores de cada hiperparâmetro e a atualiza com base na recompensa: quando a acurácia obtida supera 95% do melhor resultado histórico anterior, as probabilidades dos genes escolhidos são multiplicadas por 1,2 (reforço positivo). A amostragem segue a distribuição atualizada.

Esse modelo captura o princípio central do RL aplicado a NAS (exploração guiada por recompensa), sem a complexidade de uma rede controladora com LSTM, como no trabalho de Zoph e Le.

Simplificações em relação à literatura:

- O agente real de NAS com RL (e.g., [?]) é uma RNN que gera sequências de tokens de arquitetura e é treinada com REINFORCE. Aqui o agente é um vetor de probabilidades por hiperparâmetro.
- Não há reforço negativo explícito (punição). As probabilidades de escolhas ruins diminuem apenas relativamente, à medida que as bem-sucedidas crescem.
- As distribuições não são renormalizadas a cada passo, o que pode levar a pesos desproporcionalmente altos em runs longos.

Bug corrigido na revisão: na versão original, a condição de reforço ($acc \geq melhor_acc * 0.95$) era verificada *após* a atualização de `melhor_acc`. Como `melhor_acc` era atualizado para `acc` antes da verificação, a condição era quase sempre verdadeira, aplicando reforço indiscriminadamente. A correção move a atualização de `melhor_acc` para *após* o reforço.

```

1 def rl_search(max_evals):
2     probs = {
3         k: {v: 1.0 / len(search_space[k]) for v in search_space[k]}
4         for k in search_space
5     }
6     melhor_acc, melhor_arq = 0, None
7
8     for _ in range(max_evals):
9         arq = {
10             k: random.choices(list(probs[k].keys()),
11                               weights=list(probs[k].values()),
12                               k=1)[0]
13             for k in search_space
14         }
15         acc = avaliar_arquitetura(arq)
16
17         # Reforço: compara com melhor_acc ANTES de atualiza-lo
18         if acc >= melhor_acc * 0.95:
19             for k, v in arq.items():
20                 probs[k][v] *= 1.2
21
22         if acc > melhor_acc:
23             melhor_acc, melhor_arq = acc, arq
24
25     return melhor_arq, melhor_acc

```

Listing 4: RL como Multi-Armed Bandit por hiperparâmetro.

4.4 Bayesian Optimization (BO)

Mecanismo: utiliza um modelo substituto (*surrogate model*) para estimar a acurácia esperada de arquiteturas sem avaliá-las na VSt. O ciclo é: (1) as primeiras 3 avaliações são aleatórias (*warm-up*); (2) o surrogate é treinado com o histórico acumulado; (3) um pool de 50 candidatos aleatórios é gerado e avaliado pelo surrogate; (4) o candidato com maior acurácia prevista é submetido à VSt real; (5) o resultado é incorporado ao histórico e o ciclo se repete.

O surrogate escolhido foi o `RandomForestRegressor` (Scikit-learn), que lida naturalmente com variáveis categóricas codificadas como índices inteiros e não requer escalonamento.

Simplificações em relação à literatura:

- O surrogate canônico de BO para NAS é o *Gaussian Process* (GP), que provê incerteza explícita e permite funções de aquisição como *Expected Improvement* (EI) ou *Upper Confidence Bound* (UCB). O Random Forest não fornece incerteza calibrada.
- A estratégia de aquisição aqui é puramente exploratória: escolhe-se o candidato com maior média prevista (*exploitation* puro), sem balancear exploração/exploração.
- O SSp é codificado como vetor de índices inteiros, o que introduz ordenação artificial entre valores categóricos (e.g., `relu=0`, `elu=1`, `tanh=2`). Um encoding mais correto seria *one-hot*.

```

1 def bayesian_search(max_evals):
2     def dict_to_vec(arq):

```

```

3         return [search_space[k].index(arq[k]) for k in
               search_space]
4
5     historico_X, historico_y = [], []
6     melhor_acc, melhor_arq = 0, None
7     surrogate = RandomForestRegressor(n_estimators=10,
               random_state=42)
8
9     # Warm-up: avaliacoes aleatorias para inicializar o surrogate
10    for _ in range(min(3, max_evals)):
11        arq = gerar_aleatorio()
12        acc = avaliar_arquitetura(arq)
13        historico_X.append(dict_to_vec(arq))
14        historico_y.append(acc)
15        if acc > melhor_acc:
16            melhor_acc, melhor_arq = acc, arq
17
18    for _ in range(min(3, max_evals), max_evals):
19        surrogate.fit(historico_X, historico_y)
20        candidatos = [gerar_aleatorio() for _ in range(50)]
21        previsoes = surrogate.predict([dict_to_vec(c) for c in
               candidatos])
22        melhor_candidato = candidatos[np.argmax(previsoes)]
23        acc = avaliar_arquitetura(melhor_candidato)
24        historico_X.append(dict_to_vec(melhor_candidato))
25        historico_y.append(acc)
26        if acc > melhor_acc:
27            melhor_acc, melhor_arq = acc, melhor_candidato
28
29    return melhor_arq, melhor_acc

```

Listing 5: Bayesian Optimization com surrogate Random Forest.

4.5 Hill Climbing / Gradient-based (GO)

Mecanismo: análogo discreto da descida de gradiente. Parte de um ponto inicial aleatório no SSp. A cada passo, gera um vizinho alterando exatamente um hiperparâmetro para um valor aleatório (equivalente a um passo unitário em uma dimensão do SSp). Move-se para o vizinho somente se ele for melhor que o estado atual — aceita apenas *gradiente positivo*.

Simplificações em relação à literatura:

- Algoritmos GO reais para NAS (e.g., DARTS) operam em SSp contínuo, usando gradientes verdadeiros calculados por backpropagation sobre parâmetros de arquitetura α . Aqui o espaço é discreto e o “gradiente” é apenas a direção de melhora.
- A ausência de reinicializações torna o algoritmo susceptível a ótimos locais: uma vez preso, nunca escapa.
- Não há estratégia de *momentum*, *simulated annealing* ou qualquer mecanismo para aceitar movimentos piores com alguma probabilidade.

```

1 def hill_climbing_search(max_evals):
2     atual_arq = gerar_aleatorio()

```

```

3     atual_acc = avaliar_arquitetura(atual_arq)
4     evals = 1
5
6     while evals < max_evals:
7         vizinho_arq = atual_arq.copy()
8         param = random.choice(list(search_space.keys()))
9         vizinho_arq[param] = random.choice(search_space[param])
10
11        vizinho_acc = avaliar_arquitetura(vizinho_arq)
12        evals += 1
13
14        # Aceita somente se melhora (greedy)
15        if vizinho_acc > atual_acc:
16            atual_arq, atual_acc = vizinho_arq, vizinho_acc
17
18    return atual_arq, atual_acc

```

Listing 6: Hill Climbing discreto (análogo ao gradient-based).

5 Protocolo de Comparação

Para garantir que as diferenças de desempenho reflitam a qualidade da SSt e não do custo computacional, o protocolo impõe as seguintes restrições:

- **Orçamento igual:** todas as SSt recebem exatamente `max_evals = 10` chamadas reais à VSt.
- **Mesma VSt:** todas usam a mesma função `avaliar_arquitetura` (1 época, mesmo subconjunto de dados).
- **Mesma métrica:** acurácia de validação ao final do treinamento.
- **Mesmo SSp:** dicionário `search_space` compartilhado.

O tempo de execução é medido por SSt usando `time.time()`, mas não é usado como critério de ranqueamento — serve apenas para evidenciar que o tempo é dominado pelo treinamento (VSt), não pelo overhead da SSt em si.

6 Limitações da Implementação

As seguintes limitações devem ser consideradas ao interpretar os resultados:

1. **Orçamento de 10 avaliações é muito pequeno.** Na literatura, experimentos NAS utilizam centenas a milhares de avaliações. Com 10, o componente aleatório domina: resultados variam consideravelmente entre execuções.
2. **SSp com apenas 54 pontos.** O espaço é pequeno o suficiente para ser explorado quase completamente por RS. SSt sofisticadas (BO, EA) só se diferenciam significativamente em espaços maiores.
3. **VSt de baixa fidelidade introduz ruído.** A acurácia após 1 época é ruidosa e nem sempre prediz a acurácia ao convergir. Isso prejudica especialmente BO, cujo surrogate aprende um sinal ruidoso.

4. **Variância entre execuções.** Nenhum `random.seed` global foi fixado, portanto cada execução produz resultados ligeiramente diferentes. Para comparação científica, seria necessário múltiplas execuções com sementes fixas e análise estatística.
5. **Arquitetura fixa.** O SSp não inclui variações de profundidade, tipos de camada ou conexões. Isso limita a representatividade do experimento em relação a NAS real.

A Introdução e Fundamentação Teórica

Esta seção reproduz a parte introdutória do seminário, elaborada previamente, para manter o documento autocontido.

Introdução

O campo da Aprendizagem de Máquina (Machine Learning) revolucionou diversas áreas; no entanto, o processo manual de desenhar e otimizar modelos exige perícia especializada. Para enfrentar estes desafios, surgiu o AutoML, em que o aprendizado é feito automaticamente sem a escolha prévia de um especialista, sendo a *Neural Architecture Search* (NAS) um subcampo focado na automatização do design das arquiteturas das redes neurais.

Fundamentação Teórica

Search Space (SSp)

O Espaço de Busca (SSp) define o conjunto de todos os hiperparâmetros e arquiteturas possíveis que podem ser formados, incluindo tipos de camadas, conexões e funções de ativação. Um SSp amplo garante a presença de modelos otimizados, mas pode tornar a exploração infinita. As principais categorias de SSp são:

- **Layer-based:** Cada camada só se conecta com a sua respectiva próxima camada. Assim, o otimizador é obrigado a procurar arquiteturas em série. Um exemplo (<https://arxiv.org/abs/1812.03443>) fixou 22 camadas, cada uma escolhível entre nove opções, reduzindo o tempo de busca ao limitar as opções.
- **Hierarchical-based:** Além das conexões em série, o modelo pode ter ramificações paralelas e *skip connections*. Como o espaço amostral é aumentado, necessita-se de técnicas para reduzir o tempo de processamento (<https://arxiv.org/abs/1807.11626> e <https://arxiv.org/abs/1912.00195>).
- **Cell-based:** A unidade de seleção é uma célula que será conectada em série para criar modelos com diferentes tamanhos. As camadas em cada célula são conectadas hierarquicamente, combinando os benefícios hierárquicos com menor SSp por conta da seleção baseada em células (<https://arxiv.org/abs/1712.00559>).

Como gerenciar o tamanho do SSp de forma eficiente?

- **Adaptação dinâmica do SSp:** A abordagem mais simples é limitar manualmente a quantidade de camadas/conexões, porém os métodos mais modernos modificam o SSp durante a otimização, guiados por restrições de hardware. O FP-NAS usa aprendizado probabilístico para reduzir o SSp dinamicamente, resultando em menor uso de memória e FLOPs.
- **Uso de variáveis contínuas:** Métodos como DARTS e CP-NAS transformam o SSp discreto em espaço contínuo. Cada conexão recebe uma variável contínua, e métodos baseados em gradiente ajustam os pesos dessas conexões.
- **Problema das restrições rígidas vs. poda:** Restrições excessivas forçam o algoritmo a explorar apenas complexidades específicas. Uma alternativa mais eficiente é iniciar com células complexas e aplicar *pruning* dinâmico ao longo da busca.

- **Vantagem de limitar o SSp:** Para produção em hardware restrito e integração com MLOps, a limitação prévia evita treinamentos inviáveis de arquiteturas excessivamente complexas.

Search Strategy (SSt)

A Estratégia de Busca (SSt) determina como explorar o SSp para encontrar o melhor modelo para um determinado *dataset*. Os otimizadores mais recorrentes na literatura são:

- **Reinforcement Learning (RL):** O agente recebe recompensa/punição baseada no desempenho obtido na validação. No trabalho de Zoph e Le (<https://arxiv.org/abs/1611.01578>), o problema é definido como um agente RL que otimiza acurácia por seleção de componentes arquiteturais. Demanda muito poder computacional.
- **Evolutionary Algorithms (EA):** Baseados em princípios biológicos: população inicial de arquiteturas, avaliação de performance, evolução por seleção e crossover/mutação, critério de parada predefinido.
- **Bayesian Optimization (BO):** Utiliza modelos probabilísticos para aproximar a função objetivo e selecionar o próximo conjunto de hiperparâmetros. Eficiente quando a avaliação é cara (<https://arxiv.org/abs/1806.10282>).
- **Gradient-based optimizer (GO):** Depende do gradiente da função objetivo para atualizar os hiperparâmetros iterativamente. Computacionalmente caro pelo caráter iterativo, mas acelerado por GPUs (<https://arxiv.org/abs/1910.04465>).
- **Random Search (RS):** Método simples para problemas discretos (<https://arxiv.org/abs/1902.07638>). É o mais lento dos otimizadores e tem aplicação principalmente como baseline de validação.

Estratégia de Validação (VSt)

É a etapa que mais consome tempo nos algoritmos NAS, retornando a performance de um modelo via função objetivo e realimentando SSt e SSp.

- **Full Training:** Cada arquitetura é treinada por completo. Produz comparações exatas, mas com custo exaustivo.
- **Partial Training:** Treino por $r < n$ iterações, desde o zero (tipo 1) ou com herança de pesos (*weight sharing*, tipo 2). O tipo 2 inclui: superrede treinada uma vez com subsets como modelos menores (<https://arxiv.org/abs/1806.09055>); pesos compartilhados de modelos pequenos para um grande (<https://arxiv.org/abs/2104.00597>); compartilhamento por modelos salvos (<https://arxiv.org/abs/2004.05565>); transferência compatível com EA, como o ENAS (<https://arxiv.org/abs/1802.03268>).
- **Adaptive Training:** Treino completo do zero, mas com número de iterações determinado pela curva de aprendizado (https://dynn-icml2022.github.io/papers/paper_11.pdf).
- **No Training (zero-shot):** Métodos que pontuam arquiteturas sem treinamento, com base em propriedades do modelo e/ou dos dados (<https://arxiv.org/abs/2006.04647>).